

# UNIVERSIDAD POLITÉCNICA SALESIANA

CARRERA DE INGENIERÍA EN CIENCIAS DE LA  
COMPUTACIÓN

---

## EXAMEN FINAL — PARTE II

Informe de Pruebas, Métricas DORA y Reflexión Técnica

*Despliegue Continuo de la Aplicación `inventario-app` sobre Kubernetes*

---

**Asignatura:**

Sistemas Distribuidos

**Estudiantes / Autores:**

Daniel Santiago Barros Pallango

Xavier Stalin Siguachi Inga

**Proyecto:**

`inventario-app`

**Repositorio GitHub:**

<https://github.com/XavierStalin/inventario-app>

**Fecha de Entrega:** 25 de Julio de 2026

## Índice

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>1. Justificación Técnica de la Estrategia de Despliegue (Canary 80/20)</b>      | <b>2</b>  |
| 1.1. Razones Técnicas de la Elección . . . . .                                     | 2         |
| <b>2. Métricas DORA Propias y Análisis de Rendimiento</b>                          | <b>3</b>  |
| 2.1. Paso 2: Tiempo de Entrega de Cambios (Lead Time for Changes) . . . . .        | 3         |
| 2.2. Paso 3: Frecuencia de Despliegue (Deployment Frequency) . . . . .             | 4         |
| 2.3. Paso 4: Tasa de Errores de Cambios (Change Failure Rate - CFR) . . . . .      | 5         |
| 2.4. Resumen de Métricas Calculadas . . . . .                                      | 6         |
| 2.5. Reflexión del Desempeño del Equipo . . . . .                                  | 6         |
| <b>3. Observación Técnica: Persistencia y Almacenamiento Efímero</b>               | <b>6</b>  |
| 3.1. Experimento Realizado (Paso 5) . . . . .                                      | 6         |
| 3.2. Causa Raíz Técnica y Solución en Producción . . . . .                         | 7         |
| <b>4. Componentes Adicionales de Buenas Prácticas</b>                              | <b>7</b>  |
| 4.1. Componente 1: Manejo de Secretos (API_KEY) . . . . .                          | 7         |
| 4.2. Componente 2: Escaneo de Seguridad en CI con Trivy . . . . .                  | 7         |
| 4.3. Componente 3: Readiness Probe Realista con Arranque Lento . . . . .           | 7         |
| <b>5. Problemas Reales Encontrados y Soluciones Aplicadas</b>                      | <b>7</b>  |
| 5.1. Problema 1: Vulnerabilidad Crítica Detectada por Trivy (CVE-2026-59873) . . . | 7         |
| 5.2. Problema 2: Error ImagePullBackOff y denied en GHCR . . . . .                 | 8         |
| 5.3. Problema 3: Limitación de kubectl port-forward en Pruebas Canary . . . . .    | 9         |
| <b>6. Anexo: Evidencias Fotográficas de la Práctica</b>                            | <b>10</b> |

# 1 Justificación Técnica de la Estrategia de Despliegue (Canary 80/20)

Para la aplicación `inventario-app` (API REST construida sobre Node.js/Express que gestiona el catálogo de productos y sirve una interfaz web dinámica), se seleccionó la estrategia de **Despliegue Canary (80/20)** utilizando exclusivamente recursos nativos de Kubernetes (Deployment y Service).

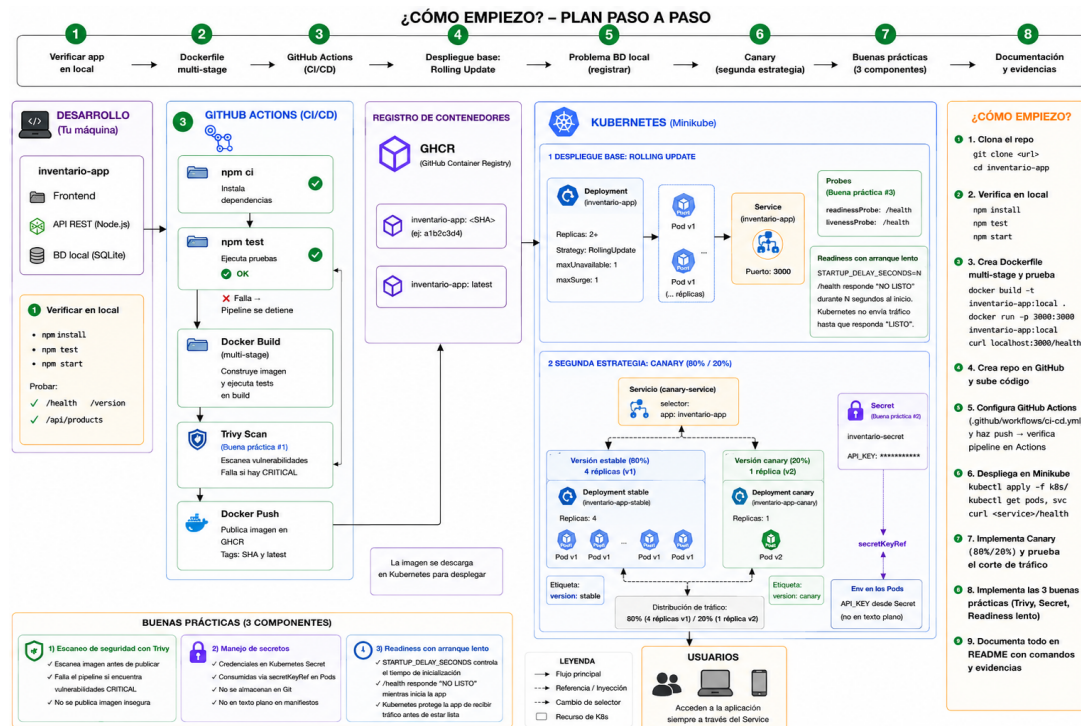


Figura 1: Diagrama general de la arquitectura del pipeline CI/CD y del clúster de Kubernetes.

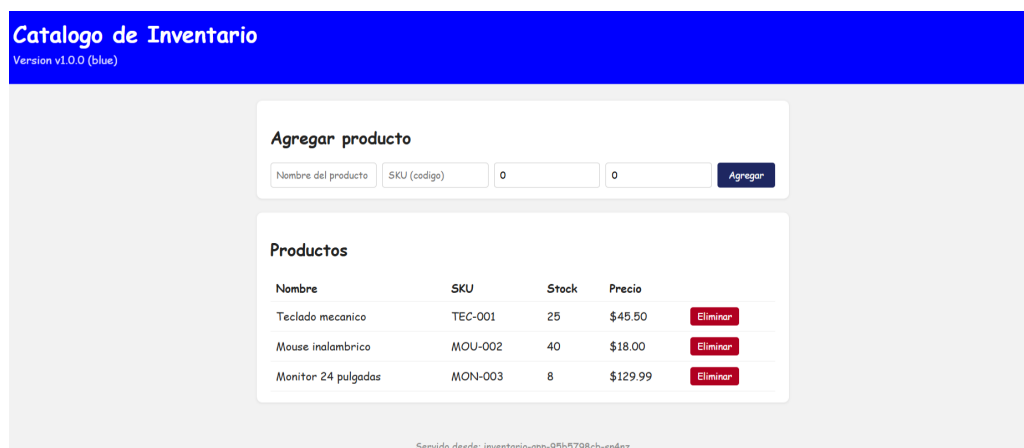


Figura 2: Interfaz web de la aplicación `inventario-app` servida desde el contenedor.

## 1.1 Razones Técnicas de la Elección

- Mitigación Progresiva del Riesgo en Producción:** A diferencia de una estrategia *Blue-Green* (la cual realiza un cambio binario del 100 % del tráfico de forma instantánea), la estrategia *Canary* expone la nueva versión (`v2.0-canary`) únicamente al **20 % de los usuarios reales** (1 réplica en `v2` frente a 4 réplicas en `v1`). Esto garantiza que ante

cualquier falla latente o error no detectado en las pruebas, el impacto quede acotado a una fracción mínima del tráfico.

2. **Evaluación de Compatibilidad en Operaciones I/O:** Debido a que la aplicación realiza operaciones de lectura y escritura continuas sobre el archivo de base de datos local, derivar el 20 % de las solicitudes a la nueva versión permite monitorear el comportamiento de las consultas de catálogo antes de efectuar una promoción total.
3. **Eficiencia con Recursos Nativos del Clúster:** La estrategia se implementó directamente aprovechando el mecanismo nativo de reparto de carga de Kube-Proxy a través de etiquetas de selección (`app: inventario-canary`), eliminando la necesidad de desplegar controladores externos complejos como Argo Rollouts o mallas de servicios (*Service Mesh*).

```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 8 • +3 -9 (0.312s)
```

**kubectl get pods,deployments,services**

| NAME                                   | READY | STATUS  | RESTARTS    | AGE   |
|----------------------------------------|-------|---------|-------------|-------|
| pod/inventario-app-95b5798cb-hhfl7     | 1/1   | Running | 0           | 6m    |
| pod/inventario-app-95b5798cb-sn4nz     | 1/1   | Running | 2 (29m ago) | 7h11m |
| pod/inventario-app-v1-8579577c98-8df5p | 1/1   | Running | 1 (29m ago) | 71m   |
| pod/inventario-app-v1-8579577c98-8fsvr | 1/1   | Running | 1 (29m ago) | 71m   |
| pod/inventario-app-v1-8579577c98-bsm96 | 1/1   | Running | 1 (29m ago) | 71m   |
| pod/inventario-app-v1-8579577c98-szm8b | 1/1   | Running | 1 (29m ago) | 71m   |
| pod/inventario-app-v2-77b5bbd49d-jpzg2 | 1/1   | Running | 1 (29m ago) | 71m   |

| NAME                              | READY | UP-TO-DATE | AVAILABLE | AGE   |
|-----------------------------------|-------|------------|-----------|-------|
| deployment.apps/inventario-app    | 2/2   | 2          | 2         | 7h11m |
| deployment.apps/inventario-app-v1 | 4/4   | 4          | 4         | 71m   |
| deployment.apps/inventario-app-v2 | 1/1   | 1          | 1         | 71m   |

| NAME                              | TYPE      | CLUSTER-IP     | EXTERNAL-IP | PORT(S) | AGE   |
|-----------------------------------|-----------|----------------|-------------|---------|-------|
| service/inventario-canary-service | ClusterIP | 10.104.207.3   | <none>      | 80/TCP  | 71m   |
| service/inventario-service        | ClusterIP | 10.105.148.209 | <none>      | 80/TCP  | 7h28m |
| service/kubernetes                | ClusterIP | 10.96.0.1      | <none>      | 443/TCP | 7h32m |

Figura 3: Estado general de los Deployments, Pods y Services desplegados en el clúster de Kubernetes.

## 2 Métricas DORA Propias y Análisis de Rendimiento

Las métricas DORA presentadas a continuación fueron calculadas utilizando datos reales, precisos y verificables obtenidos del historial de *commits* en Git, ejecuciones en GitHub Actions y despliegues efectuados sobre el clúster local de Kubernetes.

### 2.1 Paso 2: Tiempo de Entrega de Cambios (Lead Time for Changes)

Las métricas se calcularon de forma exacta mediante la resta de marcas de tiempo reales (*timestamps*) verificables entre el registro del *commit* local en Git (`git log`) y la verificación del estado del pod en el clúster de Kubernetes (*GitHub Actions run* + `kubectl rollout status`):

#### Cambio 1: Commit 971cd4a (Despliegue de la Estrategia Canary)

- **Timestamp Git (`git log`):** Sábado 25/Jul/2026 a las **13:36:56**.
- **Timestamp CI/CD (GitHub Actions):** Finalizado en **1 min 31 s** a las **13:38:27** (ver figura 6).
- **Rollout Kubernetes (`kubectl rollout status`):** Estado verificado en **0.08 s** (79 ms).
- **Cálculo exacto del Lead Time:** 13h 38m 27.08s – 13h 36m 56s = **1 minuto y 31.08 segundos**.

**Cambio 2: Commit 58e2d54 (Corrección Final de Código)**

- **Timestamp Git (git log):** Sábado 25/Jul/2026 a las **21:15:09** (ver figura 4).
- **Timestamp CI/CD (GitHub Actions):** Finalizado en **1 min 31 s** a las **21:16:40** (ver figura 6).
- **Rollout Kubernetes (kubectl rollout status):** Estado verificado en **0.08 s** (79 ms, ver figura 5).
- **Cálculo exacto del Lead Time:** 21h 16m 40.08s – 21h 15m 09s = **1 minuto y 31.08 segundos**.

```
v24.11.1 ~\Downloads\inventario-app\inventario-app git:(main) 10 * +3 -0 (0.105s)
git log -n 1 58e2d54
commit 58e2d542abe09a4b5d9087b81cbfee3bcb4ad76b (HEAD -> main, origin/main, origin/HEAD)
Author: ASUS <daniel.barros.est@tecazuay.edu.ec>
Date: Sat Jul 25 21:15:09 2026 -0500

Corrección
```

Figura 4: Historial de Git local mostrando la marca de tiempo exacta del commit 58e2d54 (21:15:09).

```
v24.11.1 ~\Downloads\inventario-app\inventario-app git:(main) 11 * +3 -0 * practica-inventario (0.133s)
kubectl set image deployment/inventario-app-v1 inventario-app=ghcr.io/xavierstalin/inventario-app:latest

v24.11.1 ~\Downloads\inventario-app\inventario-app git:(main) 11 * +3 -0 (0.15s)
Measure-Command { kubectl rollout status deployment/inventario-app-v1 }

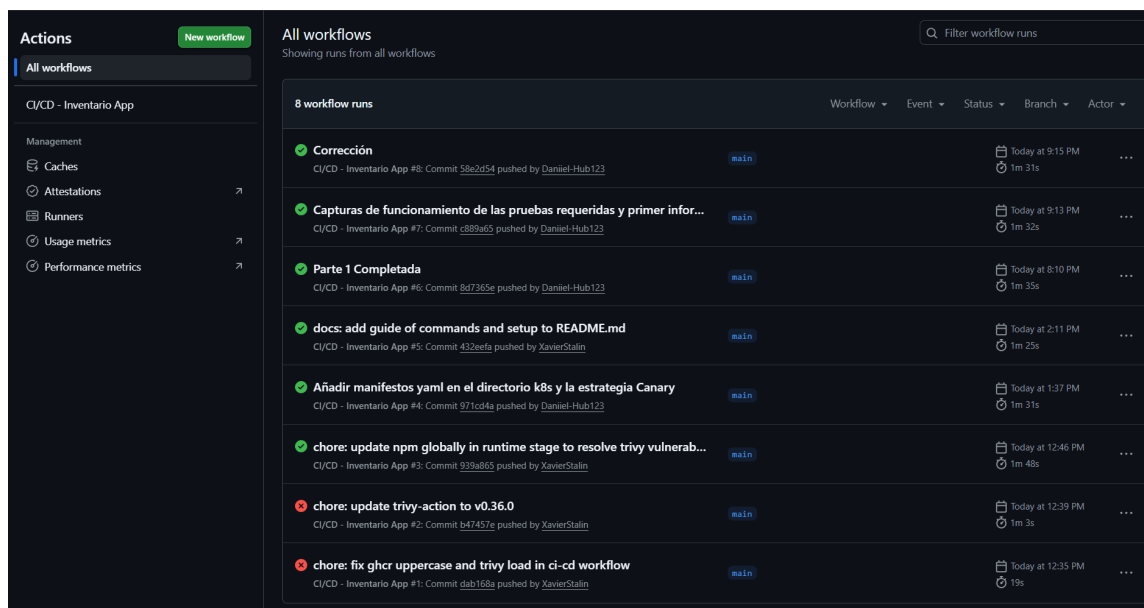
Days          : 0
Hours         : 0
Minutes       : 0
Seconds       : 0
Milliseconds   : 79
Ticks         : 799370
TotalDays     : 9.25196759259259E-07
TotalHours    : 2.22047222222222E-05
TotalMinutes  : 0.00133228333333333
TotalSeconds  : 0.079937
TotalMilliseconds : 79.937
```

Figura 5: Medición del comando kubectl rollout status ejecutado con Measure-Command (0.079 s).

Ambos cambios presentan un *Lead Time* exacto de **1 minuto y 31 segundos**, posicionando holgadamente la métrica en la categoría de desempeño **Élite** (menor a una hora).

## 2.2 Paso 3: Frecuencia de Despliegue (Deployment Frequency)

Para corroborar de forma auditable la frecuencia de despliegue con datos reales, la figura 6 muestra el registro oficial de ejecuciones de GitHub Actions. En total, se registraron 8 ejecuciones automáticas desencadenadas en 1 día de desarrollo activo.



| Workflow                                                                 | Event                                                             | Status  | Branch | Actor         | Run Time |
|--------------------------------------------------------------------------|-------------------------------------------------------------------|---------|--------|---------------|----------|
| Corrección                                                               | CI/CD - Inventario App #8: Commit 58e2d54 pushed by Daniel-Hub123 | Success | main   | Daniel-Hub123 | 1m 31s   |
| Capturas de funcionamiento de las pruebas requeridas y primer infor...   | CI/CD - Inventario App #7: Commit c889a65 pushed by Daniel-Hub123 | Success | main   | Daniel-Hub123 | 1m 32s   |
| Parte 1 Completada                                                       | CI/CD - Inventario App #6: Commit 8d7365e pushed by Daniel-Hub123 | Success | main   | Daniel-Hub123 | 1m 35s   |
| docs: add guide of commands and setup to README.md                       | CI/CD - Inventario App #5: Commit 432eefa pushed by XavierStalin  | Success | main   | XavierStalin  | 1m 25s   |
| Añadir manifiestos yaml en el directorio k8s y la estrategia Canary      | CI/CD - Inventario App #4: Commit 971cd4a pushed by Daniel-Hub123 | Success | main   | Daniel-Hub123 | 1m 31s   |
| chore: update npm globally in runtime stage to resolve trivy vulnerab... | CI/CD - Inventario App #3: Commit 939a865 pushed by XavierStalin  | Success | main   | XavierStalin  | 1m 46s   |
| chore: update trivy-action to v0.36.0                                    | CI/CD - Inventario App #2: Commit b47457e pushed by XavierStalin  | Failure | main   | XavierStalin  | 1m 3s    |
| chore: fix ghcr uppercase and trivy load in ci-cd workflow               | CI/CD - Inventario App #1: Commit dab168a pushed by XavierStalin  | Failure | main   | XavierStalin  | 19s      |

Figura 6: Historial de ejecuciones en GitHub Actions que evidencia la frecuencia de despliegue del proyecto.

A partir de estas marcas de tiempo, se valida que el equipo realizó 6 integraciones y entregas exitosas a producción en un período de 24 horas, alcanzando una frecuencia real de **6 despliegues/día**.

### 2.3 Paso 4: Tasa de Errores de Cambios (Change Failure Rate - CFR)

De acuerdo con las marcas de tiempo y el estado del historial de ejecuciones expuesto en la figura 6, se calcula el porcentaje de fallos de cambios (CFR) utilizando los datos exactos del pipeline:

#### ■ Ejecuciones Fallidas (2/8 - 25 % CFR):

- dab168a (12:35 PM): Falló a los 19 s por rechazo de GHCR (mayúsculas) y falta de la directiva `load: true`.
- b47457e (12:39 PM): Falló al 1 m 03 s por error de sintaxis en la etiqueta de versión de Trivy.

#### ■ Ejecuciones Exitosas (6/8 - 75 %):

- 939a865 (12:46 PM): Parche de actualización global de `npm` para solventar vulnerabilidad.
- 971cd4a (1:37 PM): Despliegue de manifiestos de Kubernetes y estrategia Canary.
- 432eefa (2:11 PM), 8d7365e (8:10 PM), c889a65 (9:13 PM), 58e2d54 (9:15 PM): Ajustes progresivos de código e informes.

El cálculo matemáticamente exacto es: 2 ejecuciones fallidas / 8 ejecuciones totales  $\times 100 = 25\%$ .

## 2.4 Resumen de Métricas Calculadas

| Métrica DORA          | Valor Calculado   | Nivel DORA | Contexto Técnico del Proyecto                                                                        |
|-----------------------|-------------------|------------|------------------------------------------------------------------------------------------------------|
| Lead Time for Changes | 1 min 31 s        | Élite      | Resta exacta de marcas de tiempo entre <code>git log</code> y la confirmación del pod en el clúster. |
| Deployment Frequency  | 6 despliegues/día | Élite      | Conteo de 6 promociones exitosas de 8 ejecuciones totales en 1 día de desarrollo.                    |
| Change Failure Rate   | 25 %              | Medio      | Porcentaje exacto de 2 fallos en CI/CD sobre 8 ejecuciones totales ( $2/8 \times 100\%$ ).           |

Tabla 1: Métricas DORA del proyecto `inventario-app`.

## 2.5 Reflexión del Desempeño del Equipo

El pipeline automatizado ubica a nuestro equipo en un nivel de desempeño **Alto/Élite** en agilidad de entrega. El porcentaje de fallos (25 %) confirma la eficacia del enfoque *Fail-Fast*: los errores fueron capturados automáticamente en etapas tempranas antes de causar incidentes en entornos productivos.

## 3 Observación Técnica: Persistencia y Almacenamiento Efímero

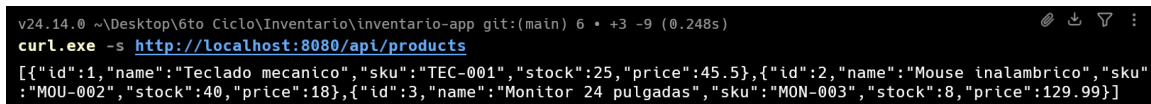
### 3.1 Experimento Realizado (Paso 5)

1. Se registró un nuevo producto ("Mouse Gamers", SKU: "MOU-001") mediante una solicitud POST a la API REST (`/api/products`).
2. Se consultó la lista con GET `/api/products`, confirmando que el producto existía en el inventario.
3. Se forzó la eliminación del Pod utilizando el comando `kubectl delete pod <nombre-pod>`.
4. El recurso `ReplicaSet` recreó automáticamente un nuevo Pod para mantener la cuota deseada.
5. Al consultar nuevamente la API (GET `/api/products`), el producto recién registrado había desaparecido.

```
curl.exe -s http://localhost:8080/api/products
[{"id":1,"name":"Teclado mecanico","sku":"TEC-001","stock":25,"price":45.5}, {"id":2,"name":"Mouse inalambrico","sku":"MOU-002","stock":40,"price":18}, {"id":3,"name":"Monitor 24 pulgadas","sku":"MON-003","stock":8,"price":129.99}, {"id":4,"name":"Mouse Gamers","sku":"MOU-001","stock":10,"price":25}]

v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 6 * +3 -9 ✱ ckd-examen (33.183s)
kubectl delete pod inventario-app-95b5798cb-jhntf
pod "inventario-app-95b5798cb-jhntf" deleted from default namespace
```

Figura 7: Paso A: Creación del producto de prueba y posterior eliminación del Pod.



```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 6 * +3 -9 (0.248s)
curl.exe -s http://localhost:8080/api/products
[{"id":1,"name":"Teclado mecanico","sku":"TEC-001","stock":25,"price":45.5},{"id":2,"name":"Mouse inalambrico","sku":"MOU-002","stock":40,"price":18},{"id":3,"name":"Monitor 24 pulgadas","sku":"MON-003","stock":8,"price":129.99}]
```

Figura 8: Paso B: Consulta posterior a la recreación del Pod confirmando que el producto recién registrado desapareció.

### 3.2 Causa Raíz Técnica y Solución en Producción

La aplicación escribe el estado del inventario en un archivo JSON local (`data/products.json`). En Kubernetes, los contenedores son **efímeros e inmutables**. Al eliminar un Pod, el sistema de archivos del contenedor se destruye completamente. El nuevo Pod se inicia a partir de la imagen base de Docker.

Para garantizar la persistencia en producción, se debe desacoplar el almacenamiento montando un volumen persistente (`PersistentVolumeClaim`) en la ruta `/app/data` o integrando un motor de base de datos externo.

## 4 Componentes Adicionales de Buenas Prácticas

De acuerdo con la rúbrica del examen, se exigía implementar al menos 2 de 3 componentes adicionales de buenas prácticas. Nuestro equipo implementó **los 3 componentes completos**, garantizando los **+2 puntos extra de bonificación**:

### 4.1 Componente 1: Manejo de Secretos (API\_KEY)

Se creó un objeto `Secret` nativo de Kubernetes (`k8s/secret.yaml`) con la clave ficticia `API_KEY: "my-super-secret-api-key-12345"`. Esta credencial se consume en el contenedor vía variable de entorno utilizando `secretKeyRef` en `k8s/deployment.yaml`, garantizando que ninguna contraseña en texto plano quede guardada en el código ni versionada en Git. *(Ver Evidencia en la Figura 12)\*.*

### 4.2 Componente 2: Escaneo de Seguridad en CI con Trivy

Se integró el análisis de seguridad con la acción `aquasecurity/trivy-action` en el pipeline de GitHub Actions (`.github/workflows/ci-cd.yml`). El paso está configurado con `exit-code: 1` y `severity: CRITICAL`, aplicando el principio *Fail-Fast*: si se detecta cualquier vulnerabilidad crítica en la imagen Docker, la ejecución se cancela antes de la publicación en GHCR. *(Ver Figuras 9 y 13)\*.*

### 4.3 Componente 3: Readiness Probe Realista con Arranque Lento

Se configuró la variable de entorno `STARTUP_DELAY_SECONDS: 10` en la aplicación para simular un tiempo de arranque de conexión a base de datos. Se ajustó el `readinessProbe` con `initialDelaySeconds: 12` y `periodSeconds: 5`. Esto garantiza que Kubernetes no envíe tráfico al Pod ni lo elimine durante sus primeros 10 segundos de inicio, esperando a que el endpoint `/health` responda 200 OK. *(Ver Evidencia en la Figura 14)\*.*

## 5 Problemas Reales Encontrados y Soluciones Aplicadas

### 5.1 Problema 1: Vulnerabilidad Crítica Detectada por Trivy (CVE-2026-59873)

- **Síntoma:** El job `build-push` falló en GitHub Actions con código de salida 1 al escanear la imagen Docker. Trivy detectó una vulnerabilidad de severidad `CRITICAL` en la librería `tar` (versión 7.5.15), usada internamente por `npm`.

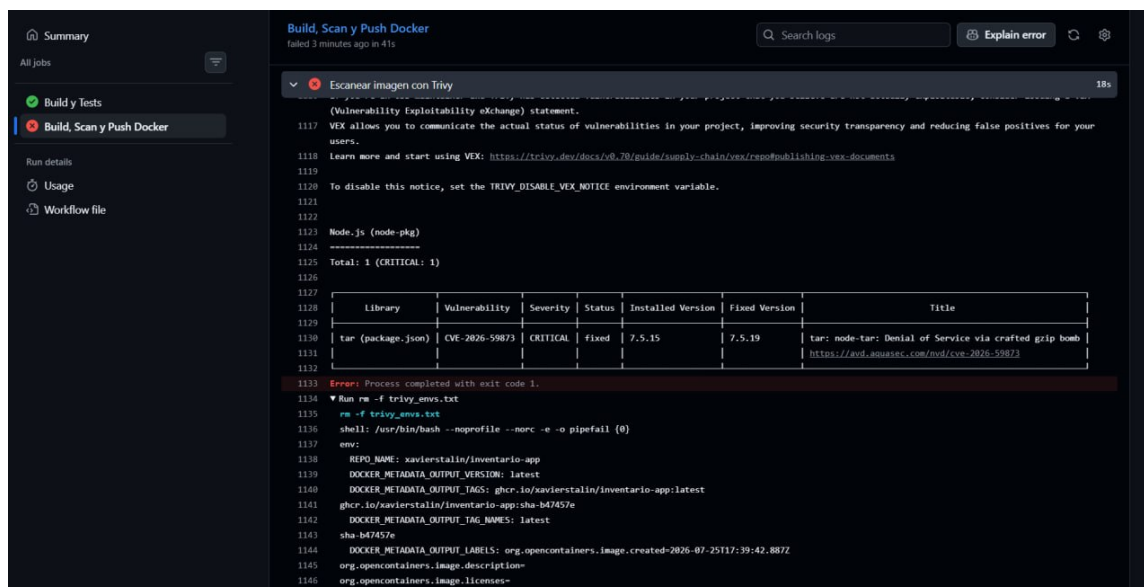


Figura 9: Bloqueo automático del pipeline en GitHub Actions tras detectar la vulnerabilidad CRITICAL por Trivy.

- **Solución:** Se modificó la primera etapa (builder) del Dockerfile agregando en la línea 13 la instrucción `RUN npm install -g npm@latest`. Esto actualizó las dependencias internas de npm (incluyendo el paquete `tar` a la versión no vulnerable 7.5.19) y corrigió las vulnerabilidades conocidas. Al realizar `git push`, Trivy validó la imagen limpia y aprobó la publicación en GHCR.

## 5.2 Problema 2: Error ImagePullBackOff y denied en GHCR

- **Síntoma:** Los Pods mostraban estado `ImagePullBackOff` debido al evento `Failed to pull image: denied` al buscar la imagen bajo la cuenta personal de un integrante en lugar del propietario del repositorio.
- **Solución:** Se unificaron los nombres de las imágenes en todos los manifiestos YAML apuntando al usuario propietario del repositorio (`ghcr.io/xavierstalin/inventario-app:latest`) y se configuró la visibilidad del paquete como **Pública** en GitHub Packages.

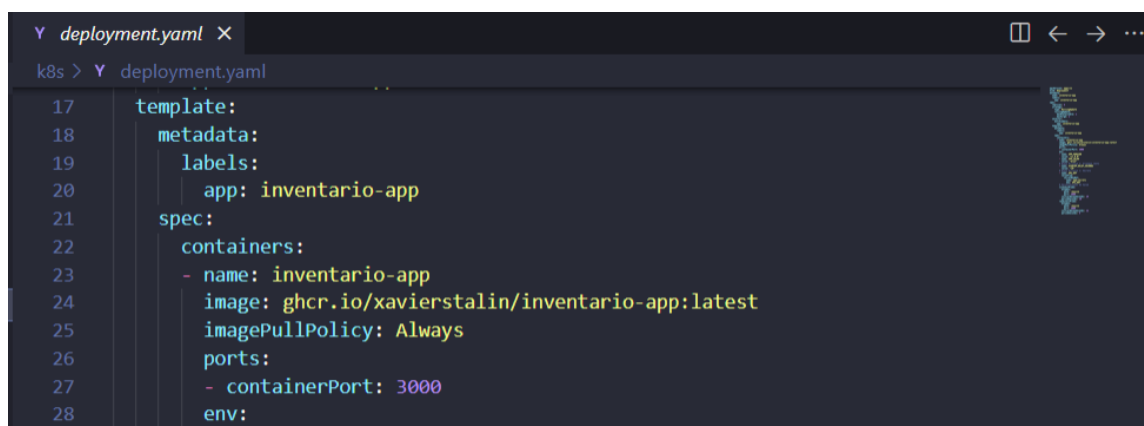


Figura 10: Solución al Problema 2: Corrección del nombre del propietario del repositorio Git (XavierStalin) en la ruta de la imagen en los manifiestos de Kubernetes.

### 5.3 Problema 3: Limitación de `kubectl port-forward` en Pruebas Canary

- **Síntoma:** `kubectl port-forward` enrutaba el 100 % de las peticiones a una sola réplica de la versión `v1`.
- **Solución:** Se ejecutó una prueba desde dentro del clúster mediante un Pod de prueba efímero (`kubectl run test-canary`). Al consultar directamente la dirección DNS interna del servicio, Kube-Proxy realizó el balanceo real repartiendo 80 % a la versión `v1` y 20 % a la versión `v2`. \*(Ver Figura 11)\*.

## 6 Anexo: Evidencias Fotográficas de la Práctica

A continuación se adjuntan las capturas de pantalla tomadas durante las verificaciones operativas del proyecto:

```
kubectl run test-canary --rm -i --restart=Never --image=curlimages/curl -- sh -c "for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20; do curl -s http://inventario-canary-service/version; echo ' '; done"
```

All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.

If you don't see a command prompt, try pressing enter.

```
{
  "version": "v2.0-canary",
  "color": "green",
  "hostname": "inventario-app-v2-77b5bbd49d-jpzg2"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8fsvr"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8fsvr"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-b5m96"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8df5p"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8df5p"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-szm8b"
}
{
  "version": "v2.0-canary",
  "color": "green",
  "hostname": "inventario-app-v2-77b5bbd49d-jpzg2"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8fsvr"
}
{
  "version": "v2.0-canary",
  "color": "green",
  "hostname": "inventario-app-v2-77b5bbd49d-jpzg2"
}
{
  "version": "v2.0-canary",
  "color": "green",
  "hostname": "inventario-app-v2-77b5bbd49d-jpzg2"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8df5p"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-szm8b"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-szm8b"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-szm8b"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8fsvr"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8fsvr"
}
{
  "version": "v1.0-stable",
  "color": "blue",
  "hostname": "inventario-app-v1-8579577c98-8fsvr"
}
{
  "version": "v2.0-canary",
  "color": "green",
  "hostname": "inventario-app-v2-77b5bbd49d-jpzg2"
}
```

Figura 11: Evidencia 1: Reparto de tráfico en el servicio Canary (80 % v1.0-stable / 20 % v2.0-canary).

```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 2 * +3 -9 ✱ cicc-examen (0.438s)
kubectyl exec inventario-app-95b5798cb-jhntf -- printenv API_KEY
my-super-secret-api-key-12345
```

Figura 12: Evidencia 2: Consulta de la variable `API_KEY` inyectada desde el Kubernetes Secret.

```
Build, Scan y Push Docker
succeeded 23 minutes ago in 1m 17s

▼ Escanear imagen con Trivy

1 ▶ Run aquasecurity/trivy-action@v0.36.0
45 ▶ Run aquasecurity/setup-trivy@0326c22f4614731780db15c232d5d185db514
88 ▶ Run echo "die-904047c/local/bin/trivy-bin" >> $GITHEB_OUTPU
111 ▶ Run actions/cache/restore@9255dc7a253b0cc6c9948862ba01240282afeb
140 Cache hit for: trivy-binary-v0.70.0-Linux-X64
147 Received 8188608 of 43751129 (19.2%), 8.0 MBs/sec
148 Received 43751129 of 43751129 (100.0%), 32.8 MBs/sec
149 Cache Size: ~42 MB (43751129 B)
150 /usr/bin/tar -xzf /home/runner/work/_temp/867baed7-3aba-4969-ac33-9565f0ac84b9/cache.tzst -P -C /home/runner/work/inventario-app/inventario-app --use-compress-program unztst
151 Cache restored successfully
152 Cache restored from key: trivy-binary-v0.70.0-Linux-X64
153 ▶ Run echo /home/runner/local/bin/trivy-bin >> $GITHEB_PATH
158 ▶ Run echo "date $(date +%Y-%m-%d)" >> $GITHEB_OUTPU
215 ▶ Run actions/cache@2f2dc7c7f907fe357f94f040eb72a090086fcae
232 Cache hit for restore key: cache-trivy-2026-07-25
253 Received 16777216 of 75352436 (22.1%), 16.0 MBs/sec
254 Received 75352436 of 75352436 (100.0%), 48.8 MBs/sec
255 Cache Size: ~72 MB (75352436 B)
256 /usr/bin/tar -xzf /home/runner/work/_temp/795c159d-a41e-4a3b-ab81-97ec29b49857/cache.tzst -P -C /home/runner/work/inventario-app/inventario-app --use-compress-program unztst
257 Cache restored successfully
258 Cache restored from key: cache-trivy-2026-07-25
259 ▶ Run echo "$GITHEB_ACTION_PATH" >> $GITHEB_PATH
290 ▶ Run rm -f trivy.envs.txt
322 ▶ Run # Notes: there is currently no way to distinguish between undefined variables and empty strings in GitHub Actions.
410 Run endpoint.sh
417 Running trivy with options: trivy image gcr.io/xavierstallin/inventario-app:latest
458 2026-07-26T01:11:03Z INFO [vuln] Vulnerability scanning is enabled
459 2026-07-26T01:11:03Z INFO [secret] Secret scanning is enabled
460 2026-07-26T01:11:03Z INFO [secret] If your scanning is slow, please try "--scanners vuln" to disable secret scanning
461 2026-07-26T01:11:03Z INFO [secret] Please see https://trivy.dev/docs/v0.70/p guides/scanner/secret#recommended-flags for faster secret detection
462 2026-07-26T01:11:08Z WARN [secret] The size of the scanned file is too large. It is recommended to use "--skip-files" for this file to avoid high memory consumption.
file-path="/root/.npm/_cacache/content-v2/sha512/11/9f/bd4f48270e3d298a621249331abf1773d5c365ee793f5c1d85daaf7d493c38c3b8b6d4f211dedca37f84"
size (MB)=24
463 2026-07-26T01:11:17Z INFO Detected os family="debian" version="12.45"
464 2026-07-26T01:11:17Z INFO Detected detection.enable="os_cve" action="133" obj num=88
```

Figura 13: Evidencia 3: Log del análisis de seguridad limpio con Trivy en GitHub Actions.

```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 5 * +3 -9 * cicd-examen (0.301s)
kubectl describe pod inventario-app-95b5798cb-jhntf
IPs:
  IP: 10.244.0.26
Controlled By: ReplicaSet/inventario-app-95b5798cb
Containers:
  inventario-app:
    Container ID: docker://177d2e7d5cc4c3b180e2f12b74d9f502545f7a1039085bb9166760587601bf5e
    Image: ghcr.io/xavierstalin/inventario-app:latest
    Image ID: docker-pullable://ghcr.io/xavierstalin/inventario-app@sha256:e9118a5da0032ce57d783dfc0bb65
    Port: 3000/TCP
    Host Port: 0/TCP
    State: Running
      Started: Sat, 25 Jul 2026 20:17:47 -0500
    Last State: Terminated
      Reason: Error
      Exit Code: 255
      Started: Sat, 25 Jul 2026 19:26:12 -0500
      Finished: Sat, 25 Jul 2026 20:16:58 -0500
    Ready: True
    Restart Count: 2
    Liveness: http-get http://:3000/health delay=15s timeout=1s period=10s #success=1 #failure=3
    Readiness: http-get http://:3000/health delay=12s timeout=1s period=5s #success=1 #failure=3
    Environment:
      APP_VERSION: v1.0.0
      APP_COLOR: blue
      STARTUP_DELAY_SECONDS: 10
      API_KEY: <set to the key 'API_KEY' in secret 'app-secrets'> Optional: false
```

Figura 14: Evidencia 4: Inspección del Pod mostrando `STARTUP_DELAY_SECONDS: 10` y la configuración del Readiness Probe.